

Joins: A join is an operation that combines the rows of two or more tables based on related columns. This operation is used for retrieving the data from multiple tables simultaneously using common columns of tables.

Types of Joins:

1.Inner Join: Inner Join is a join operation in DBMS that combines two or more tables based on related columns and returns only rows that have matching values among tables.

a) Conditional Join: Conditional join or Theta join is a type of inner join in which tables are combined based on the specified condition. In conditional join, the join condition can include $<$, $>$, $<=$, $>=$, $\neq$ operators in addition to the $=$ operator.

Table A

R	S
10	5
7	20

Table B

T	U
10	12
17	6

$$A \bowtie_{S < T} B$$

Output

R	S	T	U
10	5	10	12

(b) Equi Join: Equi Join is a type of Inner join in which we use equivalence($=$) condition in the join condition.

Table A

Column A	Column B
a	a
a	b

Table B

Column A	Column B
a	a
a	c

$$A \bowtie_{A.Column\ B = B.Column\ B} (B)$$

Output

Column A	Column B
a	a

(c) Natural Join: Natural join is a type of inner join in which we do not need any comparison operators. In natural join, columns should have the same name and domain. There should be at least one common attribute between the two tables.

Table A

Number	Square
2	4
3	9

Table B

Number	Cube
2	8
3	27

$$A \bowtie B$$

Output

Number	Square	Cube
2	4	8
3	9	27

2.Outer Join: Outer join is a type of join that retrieves matching as well as non-matching records from related tables.

a) Left Outer Join: It is also called left join. This type of outer join retrieves all records from the left table and retrieves matching records from the right table.

Table A	Table B	Output																												
<table><tr><th>Number</th><th>Square</th></tr><tr><td>2</td><td>4</td></tr><tr><td>3</td><td>9</td></tr><tr><td>4</td><td>16</td></tr></table>	Number	Square	2	4	3	9	4	16	<table><tr><th>Number</th><th>Cube</th></tr><tr><td>2</td><td>8</td></tr><tr><td>3</td><td>27</td></tr><tr><td>5</td><td>125</td></tr></table>	Number	Cube	2	8	3	27	5	125	<table><tr><th>Number</th><th>Square</th><th>Cube</th></tr><tr><td>2</td><td>4</td><td>8</td></tr><tr><td>3</td><td>9</td><td>27</td></tr><tr><td>4</td><td>16</td><td>NULL</td></tr></table>	Number	Square	Cube	2	4	8	3	9	27	4	16	NULL
Number	Square																													
2	4																													
3	9																													
4	16																													
Number	Cube																													
2	8																													
3	27																													
5	125																													
Number	Square	Cube																												
2	4	8																												
3	9	27																												
4	16	NULL																												
A ⋈ B																														

$A \bowtie B$

(b) Right Outer Join: It is also called a right join. This type of outer join retrieves all records from the right table and retrieves matching records from the left table. And for the record which doesn't lie in Left table will be marked as NULL in result Set.

$A \bowtie B$

Output:

Number	Square	Cube
2	4	8
3	9	27
5	NULL	125

(c) Full Outer Join: FULL JOIN creates the result set by combining the results of both LEFT JOIN and RIGHT JOIN. The result set will contain all the rows from both tables. For the rows for which there is no matching, the result set will contain *NULL* values.

$A \bowtie B$

Output:

Number	Square	Cube
2	4	8
3	9	27
4	16	NULL
5	NULL	125

Q.Discuss Joins in relational algebra.

Functional Dependency: In relational database management, functional dependency is a concept that specifies the relationship between two sets of attributes where one attribute determines the value of another attribute. It is denoted as $X \rightarrow Y$, where the attribute set on the left side of the arrow, X is called Determinant, and Y is called the Dependent.

Types of Functional Dependencies in DBMS:

1.Trivial Functional Dependency: In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is the subset of X, then it is called trivial functional dependency. Symbolically: $A \rightarrow B$ is trivial functional dependency if B is a subset of A. The following dependencies are also trivial: $A \rightarrow A$ & $B \rightarrow B$

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Ex: Here, $\{\text{roll_no, name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent name is a subset of determinant set $\{\text{roll_no, name}\}$. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

2.Non-trivial Functional Dependency: In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is not a subset of X, then it is called Non-trivial functional dependency.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Ex: Here, $\text{roll_no} \rightarrow \text{name}$ is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, $\{\text{roll_no, name}\} \rightarrow \text{age}$ is also a non-trivial functional dependency, since age is not a subset of $\{\text{roll_no, name}\}$

3.Multivalued Functional Dependency: In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. If $a \twoheadrightarrow \{b, c\}$ and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.

Student_ID	Course	Hobby
101	Math	Chess
101	Math	Football
101	Science	Chess
101	Science	Football

Here, **Student_ID** $\rightarrow\rightarrow$ **Course** and **Student_ID** $\rightarrow\rightarrow$ **Hobby** because:

- A student can enroll in multiple Courses.
- A student can have multiple Hobbies.
- The sets of Courses and Hobbies are independent of each other.

Thus, we have a **Multivalued Dependency**: **Student_ID** $\rightarrow\rightarrow$ [**Course**, **Hobby**]

4.Transitive Functional Dependency: In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If $a \rightarrow b$ & $b \rightarrow c$, then according to axiom of transitivity, $a \rightarrow c$. This is a transitive functional dependency.

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Ex: Here, $\text{enrol_no} \rightarrow \text{dept}$ and $\text{dept} \rightarrow \text{building_no}$.
Hence, according to the axiom of transitivity, $\text{enrol_no} \rightarrow \text{building_no}$ is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

5.Fully Functional Dependency: A Fully Functional Dependency (FFD) exists when an attribute is fully dependent on a primary key and not on any of its subsets. In other words, if we remove any part of the key, the dependency no longer holds.

Student_ID	Course_ID	Instructor_Name
101	CSE101	Dr. Smith
101	MTH102	Dr. Johnson
102	CSE101	Dr. Smith
102	PHY103	Dr. Brown

Ex: Here, the Primary Key is (**Student_ID**, **Course_ID**) because:

- A student can take multiple courses.
- A course can be taken by multiple students.
- Instructor_Name depends on both Student_ID and Course_ID.
- If we remove Course_ID, we might have multiple instructors for a student.
- If we remove Student_ID, we might have multiple students for an instructor.
- Thus, Instructor_Name is fully functionally dependent on the entire key (Student_ID, Course_ID) and not on a part of it.

Q.Define the term functional dependency and types of it.

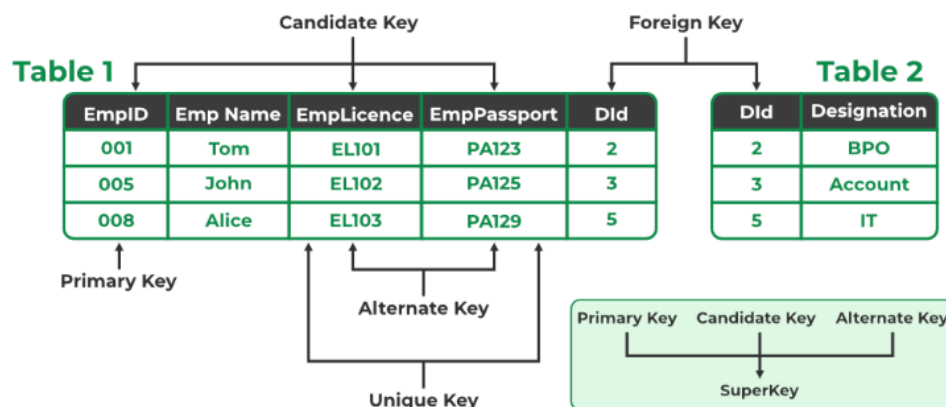
How to find candidate keys and super keys using attribute closure? Explain it with suitable example.

- 1.If attribute closure of an attribute set contains all attributes of relation, the attribute set will be super key of the relation.
- 2.If no subset of this attribute set can functionally determine all attributes of the relation, the set will be candidate key as well. For Example, using FD set of table-1.
- 3.(STUD_NO, STUD_NAME)+ = {STUD_NO, STUD_NAME, STUD_PHONE, STUD_STATE, STUD_COUNTRY, STUD_AGE}
- 4.(STUD_NO, STUD_NAME) will be super key but not candidate key because its subset.
- 5.(STUD_NO)+ is equal to all attributes of the relation. So, STUD_NO will be a candidate key.

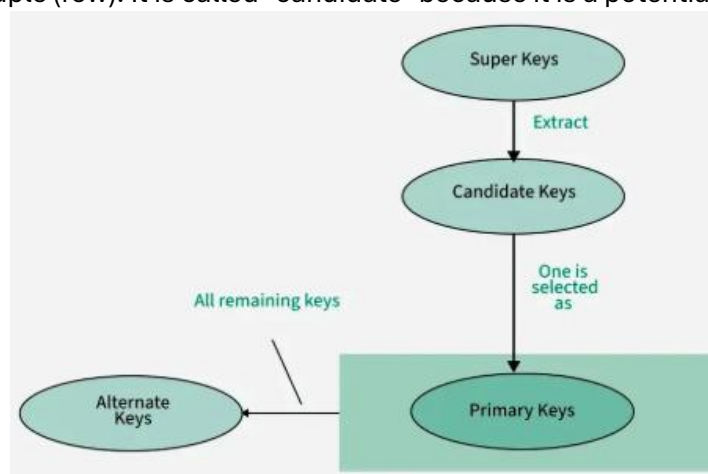
Attribute Closure: Attribute closure of an attribute set can be defined as set of attributes which can be functionally determined from it.

Prime and Non-Prime Attributes: Attributes which are parts of any candidate key of relation are called as prime attribute, others are non-prime attributes. For Example, STUD_NO in STUDENT relation is prime attribute, others are non-prime attribute.

Super Key: A Super Key in DBMS is a group of one or more attributes in a table that can uniquely identify every row in that table. It ensures no two rows have the same combination of values for those attributes.



Candidate Key: A Candidate Key is a minimal set of attributes in a relation that uniquely identifies each tuple (row). It is called "candidate" because it is a potential Primary Key.



Database Normalization: Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).

Discuss the problems related to decomposition:

Q1. Do we need to decompose a relation? (Check if it is in normal forms)

Q2. What problem (if any) does a given decomposition cause?

1. Decomposition is the process of breaking a relation into smaller relations to eliminate redundancy, maintain normalization, and improve efficiency. However, improper decomposition can lead to several problems. Below are the key issues:

2. Lossless-join property: The lossless join property in Database Management Systems (DBMS) is a crucial characteristic of a database schema decomposition. It ensures that when a relation (table) is broken down into two or more smaller relations, it's possible to perfectly reconstruct the original relation by performing a natural join operation on the decomposed relations, without losing any information or introducing spurious (incorrect) tuples.

3. Dependency-preservation property: The dependency-preservation property in Database Management Systems (DBMS) is another crucial characteristic of a database schema decomposition. It ensures that all the functional dependencies (FDs) that held true in the original relation are still enforceable in the decomposed relations, without requiring joins to check them.

Explain all the NF:

1. First Normal Form (1NF): This is the most basic level of normalization. In 1NF, each table cell should contain only a single value, and each column should have a unique name. The first normal form helps to eliminate duplicate data and simplify queries.

2. Second Normal Form (2NF): 2NF eliminates redundant data by requiring that each non-key attribute be dependent on the primary key. This means that each column should be directly related to the primary key, and not to other columns.

3. Third Normal Form (3NF): 3NF builds on 2NF by requiring that all non-key attributes are independent of each other. This means that each column should be directly related to the primary key, and not to any other columns in the same table.

4. Boyce-Codd Normal Form (BCNF): BCNF is a stricter form of 3NF that ensures that each determinant in a table is a candidate key. In other words, BCNF ensures that each non-key attribute is dependent only on the candidate key.

5. Fourth Normal Form (4NF): 4NF is a further refinement of BCNF that ensures that a table does not contain any multi-valued dependencies.

6. Fifth Normal Form (5NF): 5NF is the highest level of normalization and involves decomposing a table into smaller tables to remove data redundancy and improve data integrity.

Explain 2NF:

1. Second Normal Form (2NF) is a level of database normalization that builds upon the First Normal Form (1NF). A table is said to be in 2NF if it meets the following two conditions:

i. It must be in 1NF: This means that each cell in the table contains only a single value, there are no repeating groups of columns, and each row is uniquely identified (has a primary key).
ii. No non-prime attribute is partially dependent on any proper subset of any candidate key of the table:

- A prime attribute is any attribute that is part of at least one candidate key.
- A non-prime attribute is any attribute that is not part of any candidate key.
- Partial dependency occurs when a non-prime attribute is functionally dependent on only a part of a composite candidate key (a candidate key made up of two or more attributes), rather than the entire key.

In simpler terms: For a table to be in 2NF, every attribute that is not part of the primary key (or any other candidate key) must depend on the *entire* primary key. If a non-key attribute only depends on *part* of a composite primary key, the table is not in 2NF.

2. The main goal of achieving 2NF is to reduce data redundancy and eliminate update anomalies (insertion, deletion, and modification anomalies) that can occur when tables have partial dependencies.

- **Reduced Redundancy:** When non-key attributes depend only on a part of the key, the same information can be repeated across multiple rows. 2NF helps to separate this information into a new table where it depends on its own primary key, thus reducing duplication.
- **Improved Data Integrity:** By eliminating partial dependencies, 2NF makes it less likely that inconsistencies will arise when updating data. If information is stored in only one place (based on its full dependency), changes only need to be made there.

Explain BCNF:

1. Boyce-Codd Normal Form (BCNF) is a stricter version of Third Normal Form (3NF) that ensures a more simplified and efficient database design. It enforces that every non-trivial functional dependency must have a superkey on its left-hand side. This approach addresses potential issues with candidate keys and ensures the database is free from redundancy.

2. BCNF eliminates redundancy more effectively than 3NF by strictly requiring that all functional dependencies originate from super-keys.

3. BCNF is essential for good database schema design in higher-level systems where consistency and efficiency are important, particularly when there are many candidate keys (as one often finds with a delivery system).

4. Rules for BCNF:

Rule 1: The table should be in the 3rd Normal Form.

Rule 2: X should be a super-key for every functional dependency (FD) $X \rightarrow Y$ in a given relation.

Note: To test whether a relation is in BCNF, we identify all the determinants and make sure that they are candidate keys.

5. Example:

Given Table (Before BCNF)

We have a table with students, courses, and instructors:

Student_ID	Course	Instructor
101	Math	Dr. Smith
102	Math	Dr. Smith
103	Physics	Dr. Johnson
104	Physics	Dr. Johnson

Functional Dependencies (FDs)

1. (Student_ID, Course) → Instructor ✓ (A student taking a course has a specific instructor)
2. Instructor → Course ✗ (Problem! An instructor teaches only one course, meaning Instructor depends only on itself, not on Student_ID)

Why is this NOT in BCNF?

- The second dependency Instructor → Course is a problem because Instructor is NOT a super key.
- This means the table is not fully normalized, which can cause data redundancy.

How to Fix it? (BCNF Decomposition)

We split the table into two separate tables:

1. Instructor and Course Relationship (Who teaches what?)

- R1(Instructor, Course)

Instructor	Course
Dr. Smith	Math
Dr. Johnson	Physics

2. Student and Course Relationship (Which student takes which course?)

- R2(Student_ID, Course)

Student_ID	Course
101	Math
102	Math
103	Physics
104	Physics

Explain Relational Algebra:

1.Selection (σ): Selects tuples (rows) from a relation that satisfy a given condition.

$$\sigma_{\text{department}='IT'}(\text{Employee})$$

Result:

id	name	salary	department
2	Bob	60000	IT
4	David	55000	IT

2.Projection (π): Selects specific attributes (columns) from a relation.

$$\pi_{\text{name,salary}}(\text{Employee})$$

Result:

name	salary
Alice	50000
Bob	60000
Charlie	70000

3.Union ($\cup$): The **Union ($\cup$)** operator combines tuples from two relations (tables) while removing duplicates. Both relations must have the **same schema** (same attributes and data types). Combines all unique tuples from relations **R** and **S**.

Employee1 Table:

id	name	department
1	Alice	HR
2	Bob	IT

Employee2 Table:

id	name	department
3	Charlie	Finance
2	Bob	IT

Union Operation:

$$\text{Employee1} \cup \text{Employee2}$$

Result:

id	name	department
1	Alice	HR
2	Bob	IT
3	Charlie	Finance

4.Intersection ($\cap$): Returns a relation containing only the tuples that are common to both input relations. Returns the rows that are present in both relations R and S. Both relations must have the same schema (same attributes and data types).

Employee1 Table:

id	name	department
1	Alice	HR
2	Bob	IT
3	Charlie	Finance

Employee2 Table:

id	name	department
2	Bob	IT
3	Charlie	Finance
4	David	Sales

Intersection Operation:

$$\text{Employee1} \cap \text{Employee2}$$

Result:

id	name	department
2	Bob	IT
3	Charlie	Finance

5. Set Difference (-): Returns a relation containing tuples that are in the first relation but not in the second relation. Returns tuples that are in R but not in S. Both relations must have the same schema (same attributes and data types).

Employee1 Table:		
id	name	department
1	Alice	HR
2	Bob	IT
3	Charlie	Finance

Employee2 Table:		
id	name	department
2	Bob	IT
3	Charlie	Finance

Set Difference Operation:		
$Employee1 - Employee2$		
Result:		
id	name	department
1	Alice	HR

6. Cartesian Product (×): Combines all tuples from one relation with all tuples from another relation, creating a new relation with all possible combinations. Results in a new relation with all possible combinations of tuples from R and S. The number of resulting tuples = (number of rows in R) × (number of rows in S).

Employee Table:	
id	name
1	Alice
2	Bob

Department Table:	
dept_id	department
10	HR
20	IT

Cartesian Product Operation:			
$Employee \times Department$			
Result:			
id	name	dept_id	department
1	Alice	10	HR
1	Alice	20	IT
2	Bob	10	HR
2	Bob	20	IT

7. Rename (ρ): Renames a relation or its attributes.

Let's say we have a relation **Employee(id, name, salary)**.

Renaming **Employee** to **Workers**:

$$\rho_{Workers}(Employee)$$

- Now, **Employee** is referred to as **Workers**.

8. Join (⋈): Combines two relations based on a common attribute, creating a new relation with the combined attributes.

Notation:

$$R \bowtie S$$

Example:

$$Employees \bowtie Departments$$

- This joins **Employees** and **Departments** where they share a common attribute (e.g., **dept_id**).